Compte rendu Processus

A-1 Ouvrez une console Debian et entrez la commande pstree pour afficher sous la forme arborescente les processus.

```
Fichier  Édition  Affichage  Terminal  Onglets  Aide
root@debian:~# pstree
systemd─┬─ModemManager───2*[{ModemManager}]
        ├─NetworkManager───2*[{NetworkManager}]
        ├─agetty
        ├─alsactl
        ├─apache2───5*[apache2]
        ├─avahi-daemon───avahi-daemon
        ├─cron
        ├─dbus-daemon
        ├─exo-helper-1───xfce4-terminal─┬─bash───pstree
        │                               └─3*[{xfce4-terminal}]
```

Oui le processus systemd se trouve à la racine des processus.
C'est le processus bash qui a lancé pstree.

```
root@debian:~# pstree -p
systemd(1)─┬─ModemManager
```

Le PID de systemd est 1 car c'est le premier processus du système.

A-2 La commande ps permet d'obtenir les informations sur les processus en cours d'exécution dans la console.

```
root@debian:~# ps
  PID TTY          TIME CMD
 2134 pts/0    00:00:00 bash
 2693 pts/0    00:00:00 ps
```

Bash et ps sont en cours d'exécution. Le PID de bash est 2134 et le PID de ps est 2693.

A-3 Selon les options que nous ajoutons à ps, nous obtenons des informations sur les différents détails.

```
mika@DESKTOP-LDGKEM8:/mnt/c/WINDOWS/SystemApps/MicrosoftWindows.Client.CBS_cw5n1h2txyewy$ ps -f
UID          PID    PPID  C STIME TTY          TIME CMD
mika         524     523  0 12:22 pts/0    00:00:00 -bash
mika         700     524 99 12:27 pts/0    00:00:00 ps -f
```

La colonne UID correspond au nom d'utilisateur, PID est l'identifiant du processus, PPID est l'identifiant du processus père, C est le pourcentage de CPU utilisé, STIME est l'heure de démarrage du processus, TTY est le nom du terminal, TIME correspond à la durée d'exécution et CMD est le nom de la commande.
Le PPID de ps est 524 et le PID de bash est 524 donc c'est bash qui a lancé ps

```
mika@DESKTOP-LDGKEM8:/mnt/c/WINDOWS/SystemApps/MicrosoftWindows.Client.CBS_cw5n1h2txyewy$ ps -o pri
PRI
 19
 19
mika@DESKTOP-LDGKEM8:/mnt/c/WINDOWS/SystemApps/MicrosoftWindows.Client.CBS_cw5n1h2txyewy$ ps -o rss
  RSS
 5120
 3840
mika@DESKTOP-LDGKEM8:/mnt/c/WINDOWS/SystemApps/MicrosoftWindows.Client.CBS_cw5n1h2txyewy$ ps -o pcpu
%CPU
 0.0
 100
mika@DESKTOP-LDGKEM8:/mnt/c/WINDOWS/SystemApps/MicrosoftWindows.Client.CBS_cw5n1h2txyewy$
```

PRI est la priorité interne du processus, RSS est la quantité de mémoire RAM en Ko utilisé par le processus, %CPU est le pourcentage de CPU consommé

A-4 Toutefois, la commande ps suivi de l'option -l permet d'obtenir pas mal d'informations

```
mika@DESKTOP-LDGKEM8:/mnt/c/WINDOWS/SystemApps/MicrosoftWindows.Client.CBS_cw5n1h2txyewy$ ps -l
F S   UID     PID    PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
4 S  1000     524     523  0  80   0 -  1518 do_wai pts/0    00:00:00 bash
0 R  1000    1135     524 99  80   0 -  2078 -      pts/0    00:00:00 ps
```

La colonne UID correspond au nom d'utilisateur, PID est l'identifiant du processus, PPID est l'identifiant du processus père, C est le pourcentage de CPU utilisé, PRI correspond à la priorité, NI est la courtoisie du processus ADDR est l'adresse mémoire SZ est la taille mémoire virtuelle, WCHAN est la fonction dans laquelle le processus attend, TTY est le terminal, TIME est le temps d'exécution et CMD est le nom de la commande.

B-1 Combien de processus (tasks) sont actuellement présents dans le système ?

```
mika@DESKTOP-LDGKEM8:/mnt/c/WINDOWS/SystemApps/MicrosoftWindows.Client.CBS_cw5n1h2txyewy$ top
top - 12:54:28 up 33 min,  1 user,  load average: 0.01, 0.09, 0.04
Tasks:  23 total,   1 running,  22 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   1816.2 total,   1034.8 free,    376.4 used,    482.5 buff/cache
MiB Swap:   1024.0 total,   1024.0 free,      0.0 used.   1439.9 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
      1 root      20   0   21640  12372   9300 S   0.0   0.7   0:07.13 systemd
      2 root      20   0    3072   1920   1792 S   0.0   0.1   0:00.02 init-systemd(Ub
      6 root      20   0    3072   1792   1792 S   0.0   0.1   0:00.00 init
     48 root      19  -1   66748  15408  14384 S   0.0   0.8   0:59.98 systemd-journal
    100 root      20   0   25268   6400   4864 S   0.0   0.3   0:02.58 systemd-udevd
    132 systemd+  20   0   21456  12672  10496 S   0.0   0.7   0:00.63 systemd-resolve
    134 systemd+  20   0   91024   7680   6784 S   0.0   0.4   0:00.92 systemd-timesyn
    161 root      20   0    4236   2560   2304 S   0.0   0.1   0:00.05 cron
    162 message+  20   0    9648   4992   4480 S   0.0   0.3   0:02.62 dbus-daemon
    174 root      20   0   17960   8448   7552 S   0.0   0.5   0:01.54 systemd-logind
    177 root      20   0 1756096  12928  10752 S   0.0   0.7   0:00.93 wsl-pro-service
    179 syslog    20   0  222508   5632   4480 S   0.0   0.3   0:00.59 rsyslogd
    183 root      20   0    3160   1920   1792 S   0.0   0.1   0:00.03 agetty
    195 root      20   0    3116   1920   1792 S   0.0   0.1   0:00.06 agetty
    203 root      20   0  107032  22528  13184 S   0.0   1.2   0:00.38 unattended-upgr
    330 root      20   0    6820   4480   3712 S   0.0   0.2   0:00.03 login
    380 mika      20   0   20308  11264   9216 S   0.0   0.6   0:00.65 systemd
    381 mika      20   0   21152   3512   1792 S   0.0   0.2   0:00.00 (sd-pam)
    437 mika      20   0    6056   4864   3328 S   0.0   0.3   0:00.06 bash
    522 root      20   0    3076   1028    896 S   0.0   0.1   0:00.00 SessionLeader
    523 root      20   0    3092   1160   1024 S   0.0   0.1   0:00.07 Relay(524)
    524 mika      20   0    6072   5120   3456 S   0.0   0.3   0:00.29 bash
   1445 mika      20   0    9276   5504   3456 R   0.0   0.3   0:00.29 top
```

23 processus sont présents dans le système.

B-2 Combien sont en cours d'exécution (running) et lequel (ou lesquels) ?

1 processus est en cours d’exécution : top (affiche R (running) dans la colonne S).

B-3 Combien sont bloqués (sleeping) et lequel (ou lesquels) ?

22 processus sont bloqués :
- systemd
- init-systemd
- init
- systemd-journal
- systemd-udevd
- systemd-resolve
- system-timesyn
- cron
- dbus-daemon
- systemd-logind
- wsl-pro-service
- rsyslogd
- agetty
- agetty
- unattented-upgr
- login
- systemd
- (sd-pam)
- bash
- SessionLeader
- Relay(524)
- bash

B-4 Combien sont en état de zombie ?
0 processus est en état de zombie

B-5 Quel est le processus ancêtre de tous les autres ? Quel est le numéro (PID) de ce processus ?

Le processus ancêtre est systemd, son PID est 1.

B-6 Combien de tâches appartient au super-utilisateur (root) ? (ps aux | grep ^root | wc -l)

```
mika@DESKTOP-LDGKEM8:/mnt/c/WINDOWS/SystemApps/MicrosoftWindows.Client.CBS_cw5n1h2txyewy$ ps aux | grep ^root | wc -l
14
```

14 taches appartiennent au super-utilisateur root.

B-7 Combien appartient à l'utilisateur propriétaire de la session ? (ps aux | grep ^$LOGNAME | wc -l)

```
mika@DESKTOP-LDGKEM8:/mnt/c/WINDOWS/SystemApps/MicrosoftWindows.Client.CBS_cw5n1h2txyewy$ ps aux | grep ^$LOGNAME | wc -l
8
```

8 taches appartiennent à l’utilisateur de la session.

C-2 À l'aide de l'une des deux commandes ci-dessus, relevez les PID des deux consoles.

```
mika@DESKTOP-LDGKEM8:/mnt/c/WINDOWS/SystemApps/MicrosoftWindows.Client.CBS_cw5n1h2txyewy$ echo $$
524
```

```
mika@DESKTOP-LDGKEM8:/mnt/c/WINDOWS/SystemApps/MicrosoftWindows.Client.CBS_cw5n1h2txyewy$ echo $$
1842
```

Le PID de la première console est 524 et le PID de la deuxième console est 1842.

C-3 Entrez la commande kill -1 <Numéro du processus visé> à l'une des consoles, à partir de l'autre. Que se passe-t-il ?

Le terminal se ferme.

C-4 Dans une des deux console, entrez la commande trap -l puis relevez quelques signaux avec leur numéro.

```
mika@DESKTOP-LDGKEM8:/mnt/c/WINDOWS/SystemApps/MicrosoftWindows.mika@DESKTOP-LDGKEM8:/mnt/c/WINDOWS/SystemApps/MicrosoftWindows.Clien
t.CBS_cw5n1h2txyewy$ trap -l
 1) SIGHUP       2) SIGINT       3) SIGQUIT      4) SIGILL       5) SIGTRAP
 6) SIGABRT      7) SIGBUS       8) SIGFPE       9) SIGKILL     10) SIGUSR1
11) SIGSEGV     12) SIGUSR2     13) SIGPIPE     14) SIGALRM     15) SIGTERM
16) SIGSTKFLT   17) SIGCHLD     18) SIGCONT     19) SIGSTOP     20) SIGTSTP
21) SIGTTIN     22) SIGTTOU     23) SIGURG      24) SIGXCPU     25) SIGXFSZ
26) SIGVTALRM   27) SIGPROF     28) SIGWINCH    29) SIGIO       30) SIGPWR
31) SIGSYS      34) SIGRTMIN    35) SIGRTMIN+1  36) SIGRTMIN+2  37) SIGRTMIN+3
38) SIGRTMIN+4  39) SIGRTMIN+5  40) SIGRTMIN+6  41) SIGRTMIN+7  42) SIGRTMIN+8
43) SIGRTMIN+9  44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7
58) SIGRTMAX-6  59) SIGRTMAX-5  60) SIGRTMAX-4  61) SIGRTMAX-3  62) SIGRTMAX-2
```

```
mika@DESKTOP-LDGKEM8:/mnt/c/WINDOWS/SystemApps/MicrosoftWindows.
Client.CBS_cw5n1h2txyewy$ ps
    PID TTY          TIME CMD
   1012 pts/0    00:00:00 bash
   1057 pts/0    00:00:00 ps
mika@DESKTOP-LDGKEM8:/mnt/c/WINDOWS/SystemApps/MicrosoftWindows.
Client.CBS_cw5n1h2txyewy$ trap 'uptime' 1
mika@DESKTOP-LDGKEM8:/mnt/c/WINDOWS/SystemApps/MicrosoftWindows.
Client.CBS_cw5n1h2txyewy$ kill -1 1012
 13:34:38 up 13 min,  1 user,  load average: 0.41, 0.54, 0.32
```

La console renvoie l'heure, la durée depuis laquelle l'ordinateur est allumé, le nombre d'utilisateur connecté, la charge système moyenne.

- En utilisant la commande man uptime, donnez la fonctionnalité de celle-ci

La commande man uptime sert à afficher le manuel de la commande uptime.

C-5 Entrez cette commande dans une des fenêtres et à partir de l'autre, entrez la commande

kill -3 <Numéro du processus visé>. Vérifiez l'existence du fichier «signal3.txt» ainsi que son contenu.

```
Terminal -
Fichier  Édition  Affichage  Terminal  Onglets  Aide
root@debian:~# ps
  PID TTY          TIME CMD
 1880 pts/0    00:00:00 bash
 1885 pts/0    00:00:00 ps
root@debian:~# trap 'echo "Signal 3 reçu" >> ~/signal3.txt' 3
root@debian:~# ^\
```

```
Terminal -
Fichier  Édition  Affichage  Terminal  Onglets  Aide
root@debian:~# kill -3 1880
root@debian:~# ls
adduser.conf  Documents                Musique    signal3.txt      Vidéos
Bureau        Images                   nano.save  Téléchargements
cmdline.save  install_pilote_wifi.sh   Public     tpunix
connexion.sh  Modèles                  script.sh  typescript
root@debian:~# cat signal3.txt

Signal3 recu
Signal 3 reçu
Signal 3 reçu
Signal 3 reçu
Signal 3 reçu
Signal 3 reçu
root@debian:~#
```

D-1 Dans une des deux consoles, lancez l'éditeur de texte «nano». Dans l'autre console, entrez la commande pstree -p pour récupérer le PID de nano. Entrez la commande ls /proc/pid_nano/ et vérifiez l'existence du fichier cmdline puis relevez le contenu de celui-ci. Justifiez son contenu.

```
          ├─xfce4-terminal(1493)─┬─bash(1497)───nano(1507)
          │                      ├─bash(1505)
          │                      ├─bash(1531)───pstree(1533)
          │                      ├─{xfce4-terminal}(1494)
          │                      ├─{xfce4-terminal}(1495)
          │                      └─{xfce4-terminal}(1532)
          └─xfsettingsd(909)─┬─{xfsettingsd}(913)
                             └─{xfsettingsd}(914)
root@debian:~# ls /proc/1507/
attr             exe        mounts         projid_map    status
autogroup        fd         mountstats     root          syscall
auxv             fdinfo     net            sched         task
cgroup           gid_map    ns             schedstat     timers
clear_refs       io         numa_maps      sessionid     timerslack_ns
cmdline          limits     oom_adj        setgroups     uid_map
comm             loginuid   oom_score      smaps         wchan
coredump_filter  map_files  oom_score_adj  smaps_rollup
cpuset           maps       pagemap        stack
cwd              mem        patch_state    stat
environ          mountinfo  personality    statm
root@debian:~# cd /proc/1507
root@debian:/proc/1507# nano cmdline
```

```
  GNU nano 3.2                                                    cmdline

nano^@
```

Le fichier cmdline contient nano car c’est la commande utilisée pour lancer le programme.

E-1 Utilisez la commande ps -l pour obtenir des informations sur la priorité des processus de la console courante.

le PRI de bash et ps est 80.

E-2 Pour changer la priorité d'un processus, nous disposons de la commande nice. Tapez la commande ci-dessous. Quelles est la valeur affichée dans la colonne NI ?

La valeur NI de sleep est 10.

E-3 La commande renice permet de changer la priorité d'un processus en cours d'exécution. Tapez la commande pour diminuer la priorité du processus sleep de 5.

Le NI de sleep est passé a 5 et son PRI est passé à 85 au lieu de 90 avant.

```
root@debian:~# ps -l
F S   UID   PID  PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
0 S     0  1753  1749  0  80   0 -  1748 -      pts/0    00:00:00 bash
4 R     0  1755  1753  0  80   0 -  2636 -      pts/0    00:00:00 ps
root@debian:~# nice sleep 240 & ps -l
[1] 1758
F S   UID   PID  PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
0 S     0  1753  1749  0  80   0 -  1748 -      pts/0    00:00:00 bash
0 S     0  1758  1753  0  90  10 -  1314 hrtime pts/0    00:00:00 sleep
4 R     0  1759  1753  0  80   0 -  2636 -      pts/0    00:00:00 ps
root@debian:~# renice 5 1758
1758 (process ID) old priority 10, new priority 5
root@debian:~# ps -l
F S   UID   PID  PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
0 S     0  1753  1749  0  80   0 -  1748 -      pts/0    00:00:00 bash
0 S     0  1758  1753  0  85   5 -  1314 hrtime pts/0    00:00:00 sleep
4 R     0  1762  1753  0  80   0 -  2636 -      pts/0    00:00:00 ps
```